

Mini-Java to C

Welcome to the Compiler Course Project! In this project, you will develop a compiler to translate Mini-Java code into C, including the generation of Three Address Code (TAC). This hands-on project will help you understand the key components involved in compiler construction.

Table of Contents

- [Mini-Java to C](#)
- [Overview of Compiler Components](#)
- [Lexer \(Lexical Analysis\)](#)
- [Parser \(Syntax Analysis\)](#)
- [Semantic Analyzer](#)
- [Code Generator](#)
- [Using ANTLR](#)
- [Introduction to Mini-Java](#)
- [Supported Features of Mini-Java](#)
 - [Object-Oriented and Inheritance](#)
 - [Control Structures](#)
 - [Arrays](#)
 - [Basics](#)
- [Limitations of Mini-Java](#)
- [Understanding the Parser](#)
- [Parsing Mini-Java Code](#)
- [Steps in Parsing](#)
- [Example AST Representation](#)
- [Importance of the Parser](#)
- [Semantic Analyzer](#)
- [Scope and Variable Resolution](#)

- Example 1: Undefined Variable
- Example 2: Variable Shadowing
- Type Checking
 - Example 3: Type Mismatch
- Method Call Verification
 - Example 4: Incorrect Argument Type in Method Call
 - Example 5: Incorrect Number of Arguments in Method Call
- Resolving Instance Variables with `this`
 - Example 6: Using `this` for Accurate Scope Resolution
- Correct Usage of `break` and `continue`
 - Example 7: Ensuring `break` and `continue` are Used Within Loop Scopes
- Ensuring Effective Semantic Analysis
- Generating TAC
- `if` Statement
- `while` Loop
- `do-while` Loop
- `for` Loop
- Implementing Inheritance in C
- Understanding Inheritance in C
 - Example: Java Code with Inheritance
 - Equivalent C Code Using Structures
- Accessing Inherited Fields
- Casting Between Structures
 - Example: Casting in C
- Method Overriding Example
 - Java Example with Method Overriding
 - Translating to C with Function Pointers
 - Implementing Functions
 - Initializing Structs and Setting Up Function Pointers
 - Invoking Methods

- Handling int[] Array
- Suggestion: Predefined int_array Class
- Translating Mini-Java Array Declarations
- TAC with Functions and Stack
- Concept
- Example Translation
- Obvious Constraints and Examples
- Constraint 1: Single Entry Point
- Constraint 2: Unique Class Names
- Constraint 3: Method and Class Definition Order
- Constraint 4: Translating System.out.println()
- Constraint 5: Ignoring Comments
- Constraint 6: No Cyclic Dependencies
- Deliverables
- Timeline
- Teamwork
- Conclusion

Overview of Compiler Components

A compiler comprises four main parts:

1. **Lexer (Lexical Analysis)**
2. The lexer, or lexical analyzer, is responsible for breaking down the source Mini-Java code into tokens. These tokens are the smallest units of meaning, such as keywords, operators, identifiers, and literals. The lexer simplifies the parsing process by transforming complex strings into manageable elements. Lexical analysis in Java typically involves recognizing language-specific syntax and removing unnecessary characters like whitespace and comments.
3. **Parser (Syntax Analysis)**
4. The parser takes the tokens produced by the lexer and arranges them according to the grammatical structure of the language to build an Abstract Syntax Tree (AST). This tree representation captures the hierarchical syntactic structure of the source code, outlining the

relationships between different constructs. Parsing ensures that the code adheres to the syntax rules of Mini-Java and serves as a foundation for subsequent analysis and code generation.

5. Semantic Analyzer

6. Semantic analysis checks the source code for semantic errors, ensuring logical consistency and correctness. This phase involves type checking, scope resolution, and verifying that operations are performed on compatible data types. The semantic analyzer uses the AST to validate constraints not covered by syntax rules, such as variable declaration before use and type compatibility.

7. Code Generator

8. The code generator is responsible for translating the AST into an intermediate representation, such as Three Address Code (TAC), which simplifies the process of generating target code. After producing TAC, the code generator further translates this representation into C code, maintaining the original program's semantics. This output can be compiled and executed using a C compiler.

Using ANTLR

You can use [ANTLR](#) for this project, which simplifies the creation of the Lexer and Parser components for you. ANTLR allows you to define a grammar for Mini-Java, and it will generate the corresponding lexer and parser code.

- **Advantages of ANTLR:**
 - Reduces the complexity of manual lexer and parser implementation.
 - Accepts grammar as input and outputs the necessary code.
 - Supports multiple target languages, including Java and C++.
- **Using ANTLR:**
 - Define a well-structured grammar file for Mini-Java.
 - Use ANTLR to generate lexer and parser code from the grammar.
 - Integrate the generated code into your compiler project.

Your compiler can be implemented in any programming language, but we suggest using either C++ or Java due to their compatibility with ANTLR and prevalent use in compiler projects.

Introduction to Mini-Java

Mini-Java is a simplified version of the Java programming language designed for educational purposes, particularly to teach compiler construction. It maintains key features of Java's object-oriented nature but with several restrictions to simplify parsing and semantic analysis.

Supported Features of Mini-Java

Your compiler should support the following aspects of Mini-Java:

Object-Oriented and Inheritance

- **Classes and inheritance:** Mini-Java supports creating classes and deriving subclasses from existing ones.
- **Method overriding:** Allows a subclass to provide a specific implementation of a method that is already defined in its superclass.
- **Field access across inheritance chains:** Enables access to fields defined in superclasses.
- **this reference:** Refers to the current object whose method or constructor is being invoked.

Control Structures

- **Conditional Statements:**
 - `if`, `else if`, and `else` for branching logic.
- **Loops:**
 - `for`, `while`, and `do-while` loops for iterative operations.
 - Supports `break` and `continue` statements for loop control.

Arrays

- **Integer array support (`int[]`):** Only integer arrays are supported.
- **Array length property:** Retrieves the number of elements in an array.
- **Array indexing:** Access elements via zero-based index.

Basics

- **Basic data types:** Only `int` and `boolean` are supported as primitive types.
- **Classes and objects:** Object-oriented approach with no interfaces or inner classes.
- **Method calls:** Invoke methods defined in classes.
- **Arithmetic operations:** Supports `+`, `-`, `*`, `/`, `%` for mathematical operations.
- **Logical operations:** Supports `&&`, `||`, `!` for boolean logic.

- **Relational operators:** Includes `<`, `<=`, `>`, `>=`, `==`, `!=` for comparisons.
- **Variable assignments:** Manage and mutate data held in variables. Note that defining variable values within a class itself is not allowed (only declaration).

Limitations of Mini-Java

- No support for interfaces or inner classes.
- No support for strings or the `switch` statement.

This specification defines the scope of Mini-Java programs that your compiler will need to handle. Ensure your grammar and code generation handle these features while adhering to their constraints.

Understanding the Parser

The parser is a critical component in the compilation process, transforming the sequence of tokens generated by the lexer into a structured format, typically an Abstract Syntax Tree (AST). This representation captures the syntactic structure of the source code, facilitating semantic analysis and code generation in subsequent phases.

Parsing Mini-Java Code

Let's explore how your parser might handle and parse a simple Mini-Java program such as:

```
public class Main {  
    public static void main(String[] args) {  
        int a = 100;  
        int b = 2;  
        System.out.println(a * b);  
    }  
}
```

Steps in Parsing

1. **Tokenization:**
2. The lexer first breaks down the source code into tokens, identifying keywords like `public`, `class`, `int`, etc., identifiers like `Main`, `a`, `b`, and literals like `100` and `2`.
3. **Grammar Comprehension:**

4. The parser applies the grammar rules of Mini-Java to these tokens. It recognizes constructs such as class declarations, method definitions, variable declarations, and expressions.

5. Constructing the AST:

6. **Class Declaration:** The parser recognizes the class declaration `public class Main` and represents it as a node in the AST, with `Main` as a child node.

7. **Method Declaration:** It then identifies the `public static void main(String[] args)` method header, creating another node under the `Main` class node.

8. Variable Declarations:

- `int a = 100;` and `int b = 2;` are decomposed into type `int` with identifiers `a` and `b`, each having respective children nodes representing the literal assignments.

9. Expression Parsing:

- The expression `System.out.println(a * b);` is parsed by first identifying the method invocation `System.out.println` and then parsing the arithmetic operation `a * b`. This operation is further broken down into its respective components — identifiers `a` and `b` with the multiplication operator `*`.

10. Validation and Error Handling:

11. During parsing, the parser checks for syntactic errors. If an error is detected, such as a missing semicolon or an unexpected token, the parser provides an error message with the location and description of the error.

Example AST Representation

From the parsing process described, the AST might look something like this in a simplified representation:

- Class Node: `Main`
- Method Node: `main`
 - Declaration Node: `int a = 100`
 - Declaration Node: `int b = 2`
 - Expression Node: `System.out.println`
 - Operator Node: `*`
 - Identifier Node: `a`
 - Identifier Node: `b`

Importance of the Parser

The parser not only structures the code but also flags syntactic errors that need to be resolved for successful compilation. By creating an accurate AST, subsequent phases, like semantic analysis and code generation, receive the structured data required to enforce language semantics and produce intermediate and target code representations.

Semantic Analyzer

Semantic analysis is a crucial phase in the compilation process that ensures a program is semantically correct according to language rules. Building upon the parser's syntax tree, semantic analysis focuses on scope resolution, type checking, and enforcing semantic constraints.

Scope and Variable Resolution

Semantic analysis ensures that each variable is declared before use and that variable access is correct within the given scope. Here are some examples:

Example 1: Undefined Variable

```
int i = 100;
System.out.println(x); // Error: 'x' is not defined!
```

- **Analysis:** The parser may construct a correct syntax tree for this statement, but the semantic analysis will flag an error because `x` is used without being declared in the current or any enclosing scope.

Example 2: Variable Shadowing

```
int i = 100;
{
    boolean i = true;
    System.out.println(i); // Prints true, i is boolean
}
System.out.println(i); // Prints 100, i is integer
```

- **Analysis:** This example demonstrates variable shadowing, where the inner block's `boolean i = true;` shadows the outer `int i = 100;`. The semantic analyzer ensures that the correct variable is accessed based on the current scope.

Type Checking

Semantic analysis must enforce type constraints to ensure expressions and assignments are type-safe.

Example 3: Type Mismatch

```
int i = true; // Error: Cannot assign boolean to int
```

- **Analysis:** The semantic analysis phase detects a type mismatch error, as a boolean value `true` is being incorrectly assigned to an integer variable.

Method Call Verification

Semantic analysis checks method calls for correct argument types and counts:

Example 4: Incorrect Argument Type in Method Call

```
class A {  
    void test(int a) {}  
}  
  
A a = new A();  
a.test(true); // Error: Expected int but got boolean
```

- **Analysis:** The analyzer flags an error because the method `test` expects an `int` argument, but a `boolean` is provided.

Example 5: Incorrect Number of Arguments in Method Call

```
a.test(100, 200); // Error: Expected 1 argument but got 2
```

- **Analysis:** The semantic analyzer detects that the number of arguments provided to `test` does not match its signature, resulting in an error.

Resolving Instance Variables with `this`

The `this` keyword in Java serves to reference the current instance of a class and is crucial for resolving conflicts between class fields and local variables with overlapping names. During semantic analysis, ensuring the correct usage of `this` is essential for maintaining the integrity of the object-oriented structure.

Example 6: Using `this` for Accurate Scope Resolution

```
public class A {
```

```

int i;

public void test() {
    i = 100; // Sets the instance variable i declared in the class
    int i = 200;
    System.out.println(i); // Prints 200
    System.out.println(this.i); // Prints 100
}
}

```

- **Analysis:** In this example, the semantic analyzer must differentiate between the local variable `i` and the instance variable `i`. The use of `this.i` explicitly accesses the class field, while `i` refers to the local variable within the test method. The analyzer ensures accurate scope resolution, maintaining the proper distinctions and accesses.

Correct Usage of `break` and `continue`

The `break` and `continue` statements are used exclusively within loop constructs to either exit a loop prematurely or skip to the next iteration of the loop, respectively. Using them outside of such contexts is semantically incorrect and should be flagged during the analysis phase.

Example 7: Ensuring `break` and `continue` are Used Within Loop Scopes

```

class A {
    void test() {
        continue; // Error: Expected a loop scope

        while(true) break; // Ok: loop scope
    }
}

```

- **Analysis:** The semantic analyzer must ensure that both `break` and `continue` appear within loop constructs such as `for`, `while`, or `do-while` loops. An error should be flagged if they are used outside of these contexts.

Ensuring Effective Semantic Analysis

To implement a robust semantic analyzer, your compiler should:

- Maintain comprehensive symbol tables to track variable declarations and manage scope.
- Consistently enforce type constraints across expressions, assignments, and method calls.
- Provide clear and detailed error messages that facilitate debugging and code correction.

Generating TAC

In this section, we'll explore how to generate Three Address Code (TAC) for various control structures commonly found in programming. TAC serves as an intermediate representation, bridging the gap between the high-level program code and the final target code, typically facilitating optimization and easier translation.

if Statement

For `if` statements, TAC generation involves creating conditional and unconditional jumps to handle logical branches. As outlined previously, here's a quick recap:

Java Example:

```
if (condition) {  
    // Then block  
    ...  
} else {  
    // Else block  
    ...  
}
```

TAC Output:

```
if (!condition) goto if_else;  
if_then:  
    // Then block  
    ...  
    goto if_end;  
if_else:  
    // Else block  
    ...  
if_end:
```

while Loop

While loops require a conditional check at the beginning to decide whether the loop body should execute.

Java Example:

```
while (condition) {  
    // Loop body  
    ...  
}
```

TAC Output:

```

loop_start:
    if (!condition) goto loop_end;
    // Loop body
    ...
    goto loop_start;
loop_end:

```

do-while Loop

The `do-while` loop differs from the `while` loop in that it guarantees the loop body executes at least once before the condition is checked at the end. This feature necessitates a specific structure in the generated TAC to accommodate this initial execution prior to the condition evaluation.

Java Example:

```

do {
    // Loop body
    ...
} while (condition);

```

TAC Output:

```

do_start:
    // Loop body
    ...
    if (condition) goto do_start;

```

for Loop

The `for` loop in Java is structured to contain initialization, condition evaluation, and increment/decrement expressions, allowing it to handle loop control succinctly. Translating this into TAC involves breaking it down into distinct steps to replicate the logical flow of a `for` loop.

Java Example:

```

for (init; condition; incr) {
    // Loop body
    ...
}

```

TAC Output:

```

init;          // Initialize loop control variables
for_start:

```

```

    if (!condition) goto for_end; // Evaluate loop condition
    // Loop body
    ...
    incr; // Increment/decrement loop control variables
    goto for_start; // Repeat the loop
for_end:

```

Implementing Inheritance in C

Although C is not naturally object-oriented and does not support inheritance in the same way that Java does, we can simulate it using structures (`struct`). This section explains how you can mimic inheritance in C, which will be useful for the code generation part of your Mini-Java to C compiler project.

Understanding Inheritance in C

Inheritance is a key feature of object-oriented programming (OOP) that allows a class to inherit properties and behaviors (fields and methods) from another class. While C lacks native OOP support, we can use structures to achieve a form of inheritance-like behavior by embedding one structure within another.

Example: Java Code with Inheritance

Consider the following Java classes demonstrating simple inheritance:

```

class A {
    int a;
}

class B extends A {
    int b;
}

```

EQUIVALENT C CODE USING STRUCTURES

The same inheritance relationship can be represented in C through structures as follows:

```

struct A {
    int a;
};

struct B {
    struct A super; // Embedding structure A within B to simulate inheritance
    int b;
};

```

In this example: - `struct A` defines a single integer field `a`. - `struct B` includes `struct A` as a field named `super`, which acts like the superclass part of `B`.

Accessing Inherited Fields

To access fields from the "superclass" in a C structure, you use the `super` field, which allows you to seamlessly interact with inherited attributes. Here's how you can access `a` from an instance of `struct B`:

```
struct B *instance = malloc(sizeof(struct B));
instance->super.a = 20; // Accessing field 'a' from the "inherited" part of B
```

By embedding a parent structure within a child structure, you replicate the concept of inheritance in C. This method enables the management of hierarchical data relationships similar to classes in Java and is crucial when generating C code from Mini-Java.

By embedding a parent structure within a child structure, you replicate the concept of inheritance in C. This method enables the management of hierarchical data relationships similar to classes in Java and is crucial when generating C code from Mini-Java.

Casting Between Structures

One of the advantages of embedding a parent structure as the first field of a child structure in C is that it allows you to cast between the child and parent types effortlessly. This capability is integral to simulating polymorphism and inheritance found in object-oriented languages like Java.

Why the Order Matters

By placing the `super` field (which represents the parent structure) as the first field in `struct B`, you ensure that the memory layout of `struct A` and the beginning of `struct B` are identical. This alignment allows you to treat an instance of `struct B` as if it were an instance of `struct A`, enabling casting in C.

Example: Casting in C

Consider the following example where casting is applied:

```
// Example of casting
struct B *instanceB = malloc(sizeof(struct B));
struct A *instanceA = (struct A *)instanceB; // Cast B to A

// Accessing field 'a' directly through the casted instance
instanceA->a = 10;
```

Method Overriding Example

In object-oriented languages like Java, method overriding allows a subclass to provide a specific implementation of a method that is already defined in its superclass. This is an important feature for polymorphism and dynamic method dispatch. Here's how it can be represented in C using function pointers.

Java Example with Method Overriding

Consider the following Java classes:

```
class A {
    int a;
    int test(int d) {
        return a * d;
    }
}

class B extends A {
    @Override
    int test(int d) {
        return a + d;
    }
}
```

Translating to C with Function Pointers

To simulate method overriding in C, you can use function pointers within structures. This approach allows each class (struct) to define its own implementation of a method, mimicking the polymorphic behavior seen in Java.

Here's how you can translate the Java classes into C:

```
struct A {
    int a;

    // Function pointer for the 'test' method
    int (*function_test)(void *, int);
};

struct B {
    struct A super; // 'super' structure to simulate inheritance

    // Function pointer for the 'test' method
    int (*function_test)(void *, int);
};
```

Implementing Functions

You need to generate functions corresponding to each method implementation in your Java classes. These functions are critical for simulating dynamic method dispatch in C. Here are the example implementations:

```
int A_function_test(void *caller, int d) {
    struct A* caller_A = (struct A *) caller;
    return caller_A->a * d;
}

int B_function_test(void *caller, int d) {
    struct B* caller_B = (struct B *) caller;
    return caller_B->super.a + d;
}
```

In these functions, casting is used to convert the generic `void*` caller back to its appropriate type (`struct A` or `struct B`) before accessing the fields. This step is crucial for ensuring that the correct data members are accessed, thus maintaining the integrity of the object's structure and behavior.

Initializing Structs and Setting Up Function Pointers

Once you've implemented the necessary functions, you'll need to initialize instances of your structs and set up their function pointers. This is an essential process to ensure that each object has access to the correct method implementations, enabling polymorphic behavior.

```
struct A *instanceA = malloc(sizeof(struct A));
instanceA->function_test = A_function_test;

struct B *instanceB = malloc(sizeof(struct B));
instanceB->function_test = B_function_test;
instanceB->super.function_test = B_function_test; // Overriding the superclass
method
```

Invoking Methods

Function pointers provide a mechanism to invoke the correct method implementations dynamically, which is crucial for simulating the polymorphic behavior typical in object-oriented programming. Here's how you can make use of function pointers in practice:

```
int res = instanceB->function_test(instanceB, 100);
```

In this example, `instanceB->function_test(instanceB, 100)` dynamically calls `B_function_test`, as expected in an overridden context. This call demonstrates method

overriding, ensuring that specific behavior defined in `struct B` is executed, even when accessed via a reference with a broader class perspective.

Handling `int[]` Array

In translating Mini-Java to C, handling arrays requires careful attention since Java provides built-in functionality such as accessing the length of an array with ease. Meanwhile, C uses basic static arrays without intrinsic length properties. To address this, we outline a method to simulate Java's array length feature using a predefined structure in C.

Suggestion: Predefined `int_array` Class

To effectively handle `int[]` arrays from Mini-Java, you can define a custom structure in C that includes an explicit length field. This structure is crucial for being able to handle array length operations similarly to Java. Here's a simple example:

```
typedef struct {
    int length;
    int *data;
} int_array;

int_array *new_int_array(int size) {
    int_array *arr = (int_array *) malloc(sizeof(int_array));
    arr->length = size;
    arr->data = (int *) calloc(size, sizeof(int));
    return arr;
}
```

Translating Mini-Java Array Declarations

When translating array declarations from Mini-Java to C, it's crucial to accurately represent the Java features. Here's how you can handle this translation using a custom `int_array` structure:

- **Declaration Example:**

In Mini-Java, declaring an integer array looks like this:

```
int[] a;
```

When translated to C, it becomes:

```
int_array *a;
```

Here, `a` is declared as a pointer to an `int_array` structure, which will handle the array data and its metadata, such as length.

- **Array Initialization Example:**

An array initialization in Mini-Java, like:

```
int[] a = new int[100];
```

Translates to:

```
int_array *a = new_int_array(100);
```

This C code creates an `int_array` with a specified length of 100, allocating memory for storing 100 integers.

- **Accessing Length Example:**

If you need to access the length of the array, Mini-Java syntax:

```
a.length
```

Translates to C as:

```
a->length
```

- **Accessing Element at Index Example:**

Accessing an array element at a specific index in Mini-Java, such as:

```
a[20]
```

Translates to C as:

```
a->data[20]
```

This accesses the 21st element in the array stored within the `data` pointer of the `int_array` structure.

This accesses the `length` field directly within the `int_array` structure, allowing you to manage and use the size of arrays as you would within Java.

TAC with Functions and Stack

This part of the project is optional and delves into a more advanced aspect of compiler design.

As we know, Three Address Code (TAC) doesn't inherently support direct method calls typical of higher-level languages. In striving for a more detailed compilation process, one can bypass the automatic use of C's internal stack and instead implement a custom stack for handling function calls.

Concept

In traditional TAC, there is no direct representation for function calls and stack management. For those interested in expanding their compiler's capabilities, implementing a custom stack helps simulate the process of managing function calls, arguments, and return values at a lower level.

Example Translation

Consider the following C code snippet that defines and calls a simple function:

```
int test(int arg) {
    printf("Hello World: %d\n", arg);
    return arg * 2;
}

int main() {
    int result = test(24);
    printf("Result: %d\n", result);
    return 0;
}
```

In converting this to a system that uses a custom stack for function calls, you might create a translation that looks something like this:

- **Declare a Custom Stack:**
 - Initialize a stack to handle the function frames manually.
- **Push Arguments to Stack:**
 - Push the argument `24` and LR (the label you want to back to after function call) onto the custom stack before invoking the function.
- **Use Goto and Label for Function Invocation:**
 - Label sections of the code dedicated to functions for direct jumping using `goto`.
- **Use Goto to Return from Function:**

- Manipulate the stack to simulate returning from a function, managing the function flow manually.

Here's a conceptual view of the translated code:

```
#define STACK_SIZE 1000
void *stack[STACK_SIZE];
int stack_top = -1;

#define push_to_stack(value) stack[++stack_top] = (void *) (value)
#define pop_from_stack() stack[stack_top--]

int main() {
    goto main;

    main: {
        push_to_stack(24);
        void *lr = &main_after_test; // Simulate Link Register
        push_to_stack(lr);
        goto test;

        main_after_test:
        int result = (int) (intptr_t) pop_from_stack();
        printf("Result: %d\n", result);
        return 0;
    }

    test: {
        void *lr = pop_from_stack();
        int arg = (int) (intptr_t) pop_from_stack();
        printf("Hello World: %d\n", arg);
        push_to_stack(arg * 2);
        goto *lr;
    }
}
```

- **Analysis:** In this setup, you manually manage the stack to handle function arguments and positions via explicit `push` and `pop` operations. Function entry and exit points are controlled using labeled sections and `goto` statements, providing a clear, albeit more complex, method of simulating function calls.

Obvious Constraints and Examples

When developing your Mini-Java compiler, it's crucial to enforce certain constraints to ensure the program's correctness and reliability. Below are some key constraints that should be enforced during the compilation process, along with examples to illustrate each:

Constraint 1: Single Entry Point

A valid Mini-Java program should have only one entry point, defined with the signature `public static void main(String[] args)`.

Example of Correct Usage:

Ensure there is exactly one `main` method with the specified signature:

```
public class Main {  
    public static void main(String[] args) { // Entry point  
        System.out.println(24);  
    }  
}
```

- **Analysis:** Ensure that your compiler checks for exactly one `main` method with the specified signature. Programs with zero or multiple `main` methods should be flagged with an error.

Constraint 2: Unique Class Names

No two classes within the same program should have the same name to prevent ambiguity during type resolution and compilation.

Example of Incorrect Usage:

Attempting to declare multiple classes with the same name should lead to an error:

```
public class A {}  
public class A {} // Error: Class A is already defined
```

- **Analysis:** The semantic analyzer should flag an error whenever two classes with the same name are declared, enforcing unique identifiers for every class. (Also applies for methods and variables within a class)

Constraint 3: Method and Class Definition Order

The order of method and class definitions should not affect the program's semantics. Thus, you should allow references to classes and methods defined later in the program.

Example of Non-Sequential Definitions:

It is valid to reference a class before its definition:

```
public class B extends A {}  
public class A {}
```

- **Analysis:** The compiler should correctly handle references to `A` in class `B`, even though `A` is defined later in the file. Your compiler needs to manage these dependencies correctly through scope management and forward declarations.

Constraint 4: Translating `System.out.println()`

In Mini-Java, any call to `System.out.println(arg)` should properly translate to C's `printf` function for output.

Example Translation:

```
System.out.println(arg);
```

Should be translated to

```
printf("%d\n", arg);
```

- **Analysis:** During translation to C, ensure that `System.out.println()` calls are converted to `printf` with appropriate format specifiers to match the argument type, allowing correct console output in the translated code.

Constraint 5: Ignoring Comments

Your code should correctly ignore comments, both single-line comments (`//`) and multi-line comments (`/* ... */`), to focus only on the executable code.

Examples of Comments to Ignore:

```
// This is a single-line comment
/* This is a
   multi-line comment */
```

- **Analysis:** During lexical analysis, the lexer should remove or skip over comments, ensuring that they do not interfere with the parsing and compilation process. This step helps maintain focus on the actual code logic without the distraction of non-executable comments.

Constraint 6: No Cyclic Dependencies

Class hierarchies should not contain cycles, which can lead to infinite loops and logical errors during compilation.

Example of Cyclic Dependency (Incorrect):

```
public class A extends B {}
public class B extends A {} // Error: Cyclic inheritance detected
```

- **Analysis:** The semantic analyzer should detect cyclic inheritance and flag an error. This constraint ensures that the class inheritance hierarchy forms a directed acyclic graph (DAG), maintaining logical soundness and preventing infinite loops in inheritance.

By enforcing these constraints, you can ensure the correctness and robustness of Mini-Java programs compiled by your tool while also maintaining consistency with standard programming practices.

Deliverables

1. **ANTLR Grammar File:** The `.g4` file defining the MiniJava syntax (If using ANTLR).
2. **Parser Implementation:** Code for the parser and lexer (generated by ANTLR or your own code).
3. **Semantic Analyzer Code:** Implementation of type checking and scope resolution.
4. **TAC Generation Module:** Code that translates the parse tree into Three-Address Code in C.
5. **Documentation:** A comprehensive report detailing the design decisions, implementation steps, and testing results.

Timeline

Week	Task
1	Define MiniJava language syntax and semantics
2	Set up ANTLR and create grammar file
3	Generate parser and lexer; construct parse tree
4	Implement semantic analysis
5	Develop TAC generation module

Week	Task
6	Testing and validation of generated code
7	Finalize documentation and prepare presentation

Teamwork

Read more at [Teamwork Policy](#) page.

Conclusion

Embarking on this compiler course project provides a compelling opportunity to delve into the intricacies of language processing and compiler design. By translating Mini-Java code into C, you will gain valuable insights into how high-level programming constructs are implemented at a lower level, offering both theoretical and practical understanding.

Throughout this project, you are encouraged to explore and implement the core components of a compiler—lexical analysis, parsing, semantic analysis, and code generation. Additionally, leveraging tools such as ANTLR can simplify parts of the development process, allowing you to focus on other challenging aspects, like semantic analysis and code generation.